

C # and ASP.NET Practical Journal

(E- Copy)

UID :	2405050
Roll No :	44
Name :	Saksham Dias
Academic Year :	2026-27
Class :	TY Bsc IT

INDEX

Sr.No	Practical Name	Date	Page No
1	Introduction to C# Programming – Part 1	15/06/26	3
2	Introduction to C# Programming – Part 2	16/06/26	5
3	Basic LINQ implementation in C#	17/06/26	9
4	Date and Time in LINQ implementation in C#	25/06/26	20
5	Inheritance and hierarchy – Part 1	02/07/26	25
6	Inheritance and hierarchy – Part 2	02/07/26	28
7	Structures in C#	09/07/26	35
8	ASP .NET Razor Pages	30/07/26	42

PRACTICAL NO : 01

Title : Introduction to C# Programming – Part 1

Date : 15/06/2026

Aim: To demonstrate use of variables , loops , conditional statements and functions in C#.

Q1. Write a C# code to find the sum of digits.

Flow Diagram / Logic :

Program :

```
namespace TYIT_Rep01
{
    internal class Program
    {
        static void Main()
        {
            Console.WriteLine("Enter a number : ");
            int n = int.Parse(Console.ReadLine());
            int sum = 0;
            int m = 0;
            while (n > 0)
            {
                m = n % 10;
                sum += m;
                n = n / 10;
            }
            Console.WriteLine(" Sum = " + sum);
        }
    }
}
```

Output :

Error and Debug : None

Conclusion : We successfully demonstrated displaying sum of digits using variables and while loop.

Q2. Write a C# program to print the following pattern :

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Flow diagram / Logic : None

Program :

```
namespace TYIT_Rep01
{
    internal class Program
    {
        static void Main()
        {
            for (int i = 1; i <= 6; i++)
            {
                for (int j = 1; j < i; j++)
                {
                    Console.Write("  "+j+" ");
                }
                Console.WriteLine("\n");
            }
        }
    }
}
```

Output :

Error and Debug : None

Conclusion : We successfully demonstrated displaying the pattern using variables and while loop.

Practical No : 02

Title : Introduction to C# Programming – Part 2

Date : 16/06/2026

Aim: To demonstrate use of multi-dimensional arrays in C#.

Q1. Write a C# code to find the diagonal and border sum of a matrix

Flow Diagram / Logic: None

Program:

```
using System;
using System.ComponentModel.DataAnnotations;

namespace Test_01
{
    internal class Program
    {

        public static int max_num(int[,] mat)
        {
            int max = 0;

            for (int i = 0; i < 3; i++)
            {
                for (int j = 0; j < 3; j++)
                {
                    if (mat[i, j] >= max)
                    {
                        max = mat[i, j];
                    }
                }
            }

            return max;
        }

        public static int dsum(int[,] mat)
```

```

{
    int diagonalSum = 0;

    for (int i = 0; i < 3; i++)
    {
        diagonalSum += mat[i, i];
    }

    return diagonalSum;
}

public static int bsum(int[,] mat)
{
    int sum = 0;
    int rows = mat.GetLength(0);
    int cols = mat.GetLength(1);

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (i == 0 || i == rows - 1 || j == 0 || j == cols - 1)
            {
                sum += mat[i, j];
            }
        }
    }

    return sum;
}

static void Main(string[] args)
{
    int[,] mat1 = new int[3, 3];

```

```

int[,] mat2 = new int[3, 3];

Console.WriteLine("Enter Matrix 1:");

for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        Console.Write($"Enter element [{i},{j}]: ");
        mat1[i, j] = int.Parse(Console.ReadLine());
    }
}

Console.WriteLine("Diagonal Sum of Matrix 1 = " + dsum(mat1));
Console.WriteLine("max of Matrix 2 = " + max_num(mat1));
Console.WriteLine("\nEnter Matrix 2:");

for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        Console.Write($"Enter element [{i},{j}]: ");
        mat2[i, j] = int.Parse(Console.ReadLine());
    }
}

Console.WriteLine("Diagonal Sum of Matrix 2 = " + dsum(mat2));

Console.WriteLine("Border Sum of Matrix 2 = " + bsum(mat2));
}
}
}

```

Output :

```
Enter Matrix 1:
Enter element [0,0]: 8
Enter element [0,1]: 9
Enter element [0,2]: 4
Enter element [1,0]: 5
Enter element [1,1]: 4
Enter element [1,2]: 0
Enter element [2,0]: 7
Enter element [2,1]: 5
Enter element [2,2]: 9
Diagonal Sum of Matrix 1 = 21
max of Matrix 2 = 9

Enter Matrix 2:
Enter element [0,0]: 8
Enter element [0,1]: [ConsoleInputLine_10]
Enter element [0,2]: [ConsoleInputLine_11]
Enter element [1,0]: [ConsoleInputLine_12]
Enter element [1,1]: [ConsoleInputLine_13]
Enter element [1,2]: [ConsoleInputLine_14]
Enter element [2,0]: [ConsoleInputLine_15]
Enter element [2,1]: [ConsoleInputLine_16]
Enter element [2,2]: [ConsoleInputLine_17]
Diagonal Sum of Matrix 2 = 18
Border Sum of Matrix 2 = 44
```

Error and Debug : None

Conclusion : We successfully demonstrated displaying sum diagonals of matrix 1 and border sum of elements in matrix 2

Practical No : 03

Title : Basic LINQ implementation in C#

Date : 17/06/2026

Aim: To demonstrate implementation of LINQ technology using XML

Q1. Given a XML file of Product details. Write a C# code that fetches data from the XML file.

- (a) Display all products
- (b) Display name and price of products which have price > 55000
- (c) Display name and price of products which have price < 65000
- (d) Display name and price of products which have price < 55000 but <70000
- (e) Display name and price of products which are not 'DELL'
- (f) Display name and price of products which greater than 50,000 and not iphone
- (g) Count how many HP products are there.

Flow Diagram / Logic: None

Program :

File : productdata.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Products>
    <Product>
        <pid>101</pid>
        <pname>Samsung Refridgerator</pname>
        <price>200000</price>
        <dom>10-01-2026</dom>
    </Product>

    <Product>
        <pid>102</pid>
        <pname>Samsung Galaxy M78</pname>
        <price>560000</price>
        <dom>09-07-2026</dom>
    </Product>

    <Product>
        <pid>103</pid>
        <pname>Iphone</pname>
        <price>100000</price>
        <dom>10-10-2026</dom>
    </Product>
</Products>
```

</Product>

<Product>

<pid>104</pid>

<pname>DELL Inspiron 15</pname>

<price>55000</price>

<dom>05-01-2026</dom>

</Product>

<Product>

<pid>105</pid>

<pname>Asus VivoBook 15</pname>

<price>59000</price>

<dom>10-05-2026</dom>

</Product>

<Product>

<pid>106</pid>

<pname>Apple MacBook Air M3</pname>

<price>125000</price>

<dom>15-06-2026</dom>

</Product>

<Product>

<pid>107</pid>

<pname>Microsoft Surface Laptop 6</pname>

<price>138000</price>

<dom>20-07-2026</dom>

</Product>

```
<Product>
  <pid>108</pid>
  <pname>Samsung Galaxy Book4</pname>
  <price>82000</price>
  <dom>05-08-2026</dom>
</Product>
```

```
<Product>
  <pid>109</pid>
  <pname>MSI Modern 14</pname>
  <price>67000</price>
  <dom>12-09-2026</dom>
</Product>
```

```
<Product>
  <pid>110</pid>
  <pname>LG Gram 16</pname>
  <price>145000</price>
  <dom>10-10-2026</dom>
</Product>
```

```
</Products>
```

File : Program.cs

```
using System.Linq;
using System.Linq.Expressions;
using System.Xml.Linq;
```

```
namespace xml_demo
{
    internal class Program
```

```

{
    static void Main(string[] args)
    {
        XmlDocument pd =
XDocument.Load("C:\\Users\\SXC_Student\\source\\repos\\xml_demo\\productdata.xml");
//connects to file

        var product = pd.Elements(); //understands the tag , var is generic
data type

        //Display all products

        Console.WriteLine("----- Display All Products -----
-----");

        foreach (var item in product)
        {
            Console.WriteLine(item.ToString());
        }

        // Display name and price of products which have price > 55000

        Console.WriteLine("----- Display Name and Price Of
Products having price > 55000");

        var result = product.Elements("Product").Where(x =>
(int)x.Element("price") > 55000).Select(x =>
new
{
    Name = x.Element("pname").Value,
    Price = x.Element("price").Value,
    dom = x.Element("dom").Value

});

        foreach (var item in result) {
            Console.WriteLine(item.ToString());
        }
    }
}

```

```

        // Display name and price of products which have price < 65000
        Console.WriteLine("----- Display Name and Price Of
Products having price < 65000");

        var result1 = product.Elements("Product").Where(x =>
(int)x.Element("price") < 65000).Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value

        }

        );
        foreach (var item in result1)
        {
            Console.WriteLine(item.ToString());
        }

        // Display name and price of products which have price < 55000 but
<70000

        Console.WriteLine("----- Display Name and Price Of
Products having price price < 55000 but <70000");

        var result2 = product.Elements("Product").Where(x =>
(int)x.Element("price") > 55000 && (int)x.Element("price") < 70000).Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value

        }

        );

```

```

        foreach (var item in result2)
        {
            Console.WriteLine(item.ToString());
        }

        // Display name and price of products which are not 'DELL'
        Console.WriteLine("----- Display name and price of
products which are not 'DELL'");

        var result3 = product.Elements("Product").Where(x =>
(string)x.Element("pname") != "DELL").Select(x =>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,
            dom = x.Element("dom").Value

        }

    );
    foreach (var item in result3)
    {
        Console.WriteLine(item.ToString());
    }

    // Display name and price of products which greater than 50,000 and not
iphone
    Console.WriteLine("----- Display name and price of
products which greater than 50,000 and not iphone");

    var result4 = product.Elements("Product").Where(x =>
(int)x.Element("price") > 50000 && (string)x.Element("pname") != "Iphone").Select(x
=>
        new
        {
            Name = x.Element("pname").Value,
            Price = x.Element("price").Value,

```

```

        dom = x.Element("dom").Value

    }

};

foreach (var item in result4)
{
    Console.WriteLine(item.ToString());
}

// Count how many HP products are there
Console.WriteLine("----- Count how many HP products are
there -----");

int count = 0;

var result5 = product.Elements("Product").GroupBy(x =>
(string)x.Element("pname") == "%HP%");

foreach (var item in result5)
{
    count++;
}

Console.WriteLine(count);

var pr = from x in product.Elements("Product")
        where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-
yyyy", null) > DateTime.ParseExact("10-01-2026", "dd-MM-yyyy", null)
        let nm = (string)x.Element("pname")
        select nm;

Console.WriteLine("*****");

Console.WriteLine("Q. Show all product greater than 10-01-2026 manu
date");

foreach (var item in pr)
{

```

```

        Console.WriteLine(item);
    }

    Console.WriteLine("*****");

    Console.WriteLine("Q. 5");

    var pr2 = from x in product.Elements("Product") where
        ((int)x.Element("price") > 40000) && DateTime.ParseExact((string)x.Element("dom"),
        "dd-MM-yyyy", null) > DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
        select new
        {
            pname = (string)x.Element("pname"),
            price = (int)x.Element("price"),
            dom = (string)x.Element("dom")
        };

    foreach (var item in pr2)
    {
        Console.WriteLine(item);
    }

    Console.WriteLine("*****");

    Console.WriteLine("Q. 6 ");

    var pr3 = from x in product.Elements("Product")
        where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-
yyyy", null) >
        DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
        && DateTime.ParseExact((string)x.Element("dom"), "dd-MM-
yyyy", null) <
        DateTime.ParseExact("01-10-2026", "dd-MM-yyyy", null)
        select new

```

```

        {
            pname = (string)x.Element("pname"),
            price = (int)x.Element("price"),
            dom = (string)x.Element("dom")
        };

foreach (var item in pr3)
{
    Console.WriteLine(item);
}

Console.WriteLine("*****");

    }
}
}

```

Output :

(a)

```
----- Display All Products -----
<Products>
  <Product>
    <pid>101</pid>
    <pname>Samsung Refridgerator</pname>
    <price>200000</price>
    <dom>10-01-2026</dom>
  </Product>
  <Product>
    <pid>102</pid>
    <pname>Samsung Galaxy M78</pname>
    <price>560000</price>
    <dom>09-07-2026</dom>
  </Product>
  <Product>
    <pid>103</pid>
    <pname>Iphone</pname>
    <price>100000</price>
    <dom>10-10-2026</dom>
  </Product>
  <Product>
    <pid>104</pid>
    <pname>DELL Inspiron 15</pname>
    <price>55000</price>
    <dom>05-01-2026</dom>
  </Product>
  <Product>
    <pid>105</pid>
    <pname>Asus VivoBook 15</pname>
    <price>59000</price>
    <dom>10-05-2026</dom>
  </Product>
  <Product>
    <pid>106</pid>
    <pname>Apple MacBook Air M3</pname>
    <price>125000</price>
    <dom>15-06-2026</dom>
  </Product>
  <Product>
    <pid>107</pid>
    <pname>Microsoft Surface Laptop 6</pname>
    <price>138000</price>
    <dom>20-07-2026</dom>
  </Product>
  <Product>
    <pid>108</pid>
    <pname>Samsung Galaxy Book4</pname>
    <price>82000</price>
    <dom>05-08-2026</dom>
  </Product>
  <Product>
    <pid>109</pid>
    <pname>MSI Modern 14</pname>
    <price>67000</price>
    <dom>12-09-2026</dom>
  </Product>
  <Product>
    <pid>110</pid>
    <pname>LG Gram 16</pname>
    <price>145000</price>
    <dom>10-10-2026</dom>
  </Product>
</Products>
```

(b)

```
----- Display Name and Price Of Products having price > 55000
{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = Iphone, Price = 100000, dom = 10-10-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }
```

(c)

```
----- Display Name and Price Of Products having price < 65000
{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
```

(d)

```
----- Display Name and Price Of Products having price price < 55000 but <70000
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
```

(e)

```
----- Display name and price of products which are not 'DELL'
{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = Iphone, Price = 100000, dom = 10-10-2026 }
{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }
```

(f)

```
----- Display name and price of products which greater than 50,000 and not iphone
{ Name = Samsung Refridgerator, Price = 200000, dom = 10-01-2026 }
{ Name = Samsung Galaxy M78, Price = 560000, dom = 09-07-2026 }
{ Name = DELL Inspiron 15, Price = 55000, dom = 05-01-2026 }
{ Name = Asus VivoBook 15, Price = 59000, dom = 10-05-2026 }
{ Name = Apple MacBook Air M3, Price = 125000, dom = 15-06-2026 }
{ Name = Microsoft Surface Laptop 6, Price = 138000, dom = 20-07-2026 }
{ Name = Samsung Galaxy Book4, Price = 82000, dom = 05-08-2026 }
{ Name = MSI Modern 14, Price = 67000, dom = 12-09-2026 }
{ Name = LG Gram 16, Price = 145000, dom = 10-10-2026 }
```

(g)

```
----- Count how many HP products are there -----
1
*****
```

Error and Debug : None

Conclusion : We successfully demonstrated basic implementation of LINQ technology via integrating a XML local database

Practical No : 04

Title : Date and Time in LINQ implementation in C#

Date : 25/06/2026

Aim: To demonstrate implementation of LINQ technology using XML (fetching date and time fields)_

Q1. Given a XML file of Product details. Write a C# code that fetches data from the XML file.

- (a) Show all product greater than 10-01-2026 menu date
- (b) Show all products where date is greater than 01-07-2026 and price is greater Rs 40000.
- (c) Show all products between date 01-07-2026 and 01-10-2026.

Flow Diagram / Logic: None

Program :

File : productdata.xml

```
<?xml version="1.0" encoding="utf-8" ?>
```

```
<Products>
```

```
    <Product>
```

```
        <pid>101</pid>
```

```
        <pname>Samsung Refridgerator</pname>
```

```
        <price>200000</price>
```

```
        <dom>10-01-2026</dom>
```

```
    </Product>
```

```
    <Product>
```

```
        <pid>102</pid>
```

```
        <pname>Samsung Galaxy M78</pname>
```

```
        <price>560000</price>
```

```
        <dom>09-07-2026</dom>
```

```
    </Product>
```

```
    <Product>
```

```
        <pid>103</pid>
```

```
        <pname>Iphone</pname>
```

```
        <price>100000</price>
```

```
        <dom>10-10-2026</dom>
```

```
    </Product>
```

```
<Product>
  <pid>104</pid>
  <pname>DELL Inspiron 15</pname>
  <price>55000</price>
  <dom>05-01-2026</dom>
</Product>
```

```
<Product>
  <pid>105</pid>
  <pname>Asus VivoBook 15</pname>
  <price>59000</price>
  <dom>10-05-2026</dom>
</Product>
```

```
<Product>
  <pid>106</pid>
  <pname>Apple MacBook Air M3</pname>
  <price>125000</price>
  <dom>15-06-2026</dom>
</Product>
```

```
<Product>
  <pid>107</pid>
  <pname>Microsoft Surface Laptop 6</pname>
  <price>138000</price>
  <dom>20-07-2026</dom>
</Product>
```

```
<Product>
  <pid>108</pid>
```

```

        <pname>Samsung Galaxy Book4</pname>
        <price>82000</price>
        <dom>05-08-2026</dom>
    </Product>

```

```

    <Product>
        <pid>109</pid>
        <pname>MSI Modern 14</pname>
        <price>67000</price>
        <dom>12-09-2026</dom>
    </Product>

```

```

    <Product>
        <pid>110</pid>
        <pname>LG Gram 16</pname>
        <price>145000</price>
        <dom>10-10-2026</dom>
    </Product>

```

```

</Products>

```

File : Program.cs

```

var pr = from x in product.Elements("Product")
        where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null) >
DateTime.ParseExact("10-01-2026", "dd-MM-yyyy", null)
        let nm = (string)x.Element("pname")
        select nm;

Console.WriteLine("*****");
Console.WriteLine("Q. Show all product greater than 10-01-2026 manu date");

foreach (var item in pr)
{
    Console.WriteLine(item);
}

```

```
}
```

```
Console.WriteLine("*****");
```

```
Console.WriteLine("Q. 5");
```

```
var pr2 = from x in product.Elements("Product")
          where ((int)x.Element("price") > 40000) &&
DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null) >
DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
          select new
{
    pname = (string)x.Element("pname"),
    price = (int)x.Element("price"),
    dom = (string)x.Element("dom")
};
```

```
foreach (var item in pr2)
{
    Console.WriteLine(item);
}
```

```
Console.WriteLine("*****");
```

```
Console.WriteLine("Q. 6 ");
```

```
var pr3 = from x in product.Elements("Product")
          where DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null)
          >
          DateTime.ParseExact("01-07-2026", "dd-MM-yyyy", null)
          && DateTime.ParseExact((string)x.Element("dom"), "dd-MM-yyyy", null) <
          DateTime.ParseExact("01-10-2026", "dd-MM-yyyy", null)
          select new
```

```

        {
            pname = (string)x.Element("pname"),
            price = (int)x.Element("price"),
            dom = (string)x.Element("dom")
        };

foreach (var item in pr3)
{
    Console.WriteLine(item);
}

Console.WriteLine("*****");
}
}
}

```

Output :

(a)

```

Q. Show all product greater than 10-01-2026 manu date
Samsung Galaxy M78
Iphone
Asus VivoBook 15
Apple MacBook Air M3
Microsoft Surface Laptop 6
Samsung Galaxy Book4
MSI Modern 14
LG Gram 16
*****

```

(b)

```

*****
Q. 5
{ pname = Samsung Galaxy M78, price = 560000, dom = 09-07-2026 }
{ pname = Iphone, price = 100000, dom = 10-10-2026 }
{ pname = Microsoft Surface Laptop 6, price = 138000, dom = 20-07-2026 }
{ pname = Samsung Galaxy Book4, price = 82000, dom = 05-08-2026 }
{ pname = MSI Modern 14, price = 67000, dom = 12-09-2026 }
{ pname = LG Gram 16, price = 145000, dom = 10-10-2026 }
*****

```

(c)

```

Q. 6
{ pname = Samsung Galaxy M78, price = 560000, dom = 09-07-2026 }
{ pname = Microsoft Surface Laptop 6, price = 138000, dom = 20-07-2026 }
{ pname = Samsung Galaxy Book4, price = 82000, dom = 05-08-2026 }
{ pname = MSI Modern 14, price = 67000, dom = 12-09-2026 }

```

Error and Debug : None

Conclusion : We successfully demonstrated basic implementation of LINQ technology via integrating a XML local database to fetch data based on date and time.

Practical No : 05

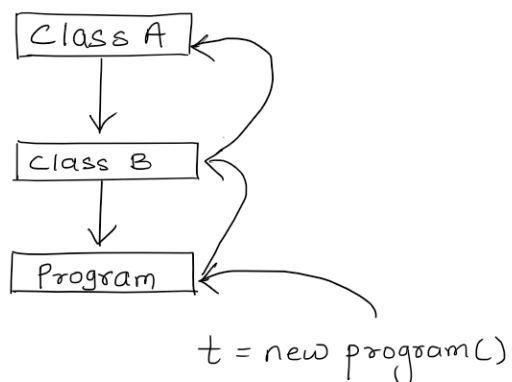
Title : Inheritance and hierarchy – Part 1

Date : 02/07/2026

Aim: To understand how constructor works

Q1. Write a C# code to demonstrate working of constructor and hierarchy.

Flow Diagram / Logic:



Program :

```
namespace Inheritance
{

    class A
    {
        public A()
        {
            Console.WriteLine("In A default");
        }

        public A(string n) : this()
        {
            Console.WriteLine("In a parameterised");
        }
    }

}
```

```
class B : A
{
    public B() : base("trial")
    {
        Console.WriteLine("In B default");
    }

    public B(string n)
    {
        Console.WriteLine("In B Parameterised");
    }

}

class Program : B
{
    static void Main(string[] args)
    {
        Program t = new Program();
    }
}
```

Output :

```
In A default
In a parameterised
In B default

C:\Users\SXC_Student\source\repos\Inheritance\bin\Deb
To automatically close the console when debugging sto
le when debugging stops.
Press any key to close this window . . .
```

Error and Debug : None

Conclusion : We successfully demonstrated how constructor works and types of constructor.

Practical No : 06

Title : Inheritance and hierarchy – Part 2

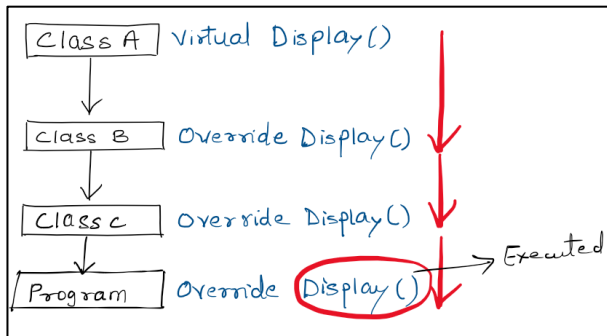
Date : 02/07/2026

Aim: To understand how inheritance works (override and virtual)

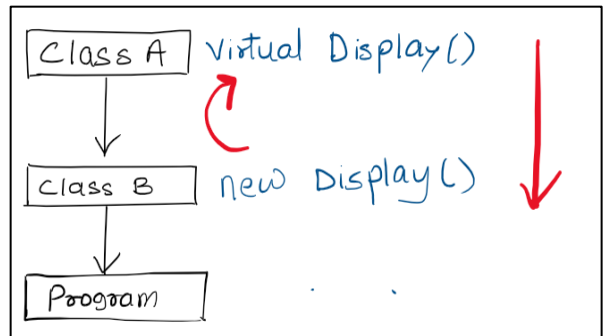
Q1. Write at-least four examples, demonstrating inheritance in C#.

Flow Diagram / Logic:

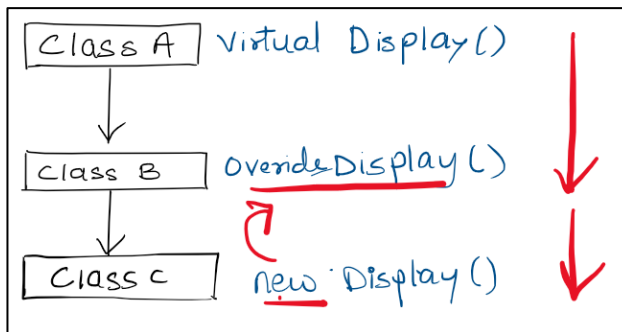
Example 1 :



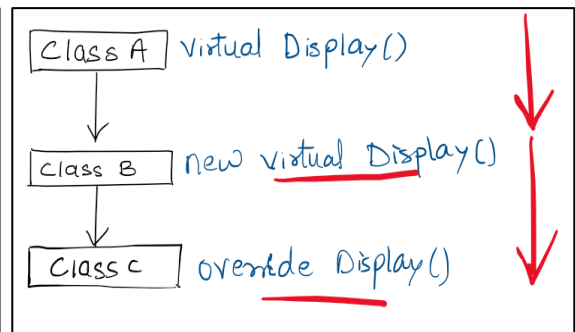
Example 2 :



Example 3 :



Example 4 :



Program :

Example 1 :

```
namespace Inheritance
{

    class A
    {
        public virtual void Display()
        {
            Console.WriteLine("In A Display");
        }
    }
}
```

```

    }

    class B : A
    {
        public override void Display()
        {
            Console.WriteLine("In B Display");
        }
    }

    class C : B
    {
        public override void Display()
        {
            Console.WriteLine("In C Display");
        }
    }

    class Program : C
    {
        public override void Display()
        {
            Console.WriteLine("In Program Display");
        }

        static void Main(string[] args)
        {
            A a = new Program();
            a.Display();
        }
    }
}

```

Example 2 :

```
namespace Inheritance
{

    class A
    {
        public virtual void Display()
        {
            Console.WriteLine("In A Display");
        }
    }

    class B : A
    {
        public new void Display()
        {
            Console.WriteLine("In B Display");
        }
    }

    class Program
    {

        static void Main(string[] args)
        {

            A a = new B();
            a.Display();
        }
    }
}
```

Example 3:

```
namespace Inheritance
{

    class A
    {
        public virtual void Display()
        {
            Console.WriteLine("In A Display");
        }
    }

    class B : A
    {
        public override void Display()
        {
            Console.WriteLine("In B Display");
        }
    }

    class C : B
    {
        public new void Display()
        {
            Console.WriteLine("In C Display");
        }
    }

    class Program
    {

        static void Main(string[] args)
        {
```

```

        A a = new C();
        a.Display();
    }
}

```

Example 4 :

`namespace` Inheritance

```

{

    class A
    {
        public virtual void Display()
        {
            Console.WriteLine("In A Display");
        }
    }

    class B : A
    {
        public new virtual void Display()
        {
            Console.WriteLine("In B Display");
        }
    }

    class C : B
    {
        public override void Display()
        {
            Console.WriteLine("In C Display");
        }
    }

    class Program
    {

```

```

static void Main(string[] args)
{

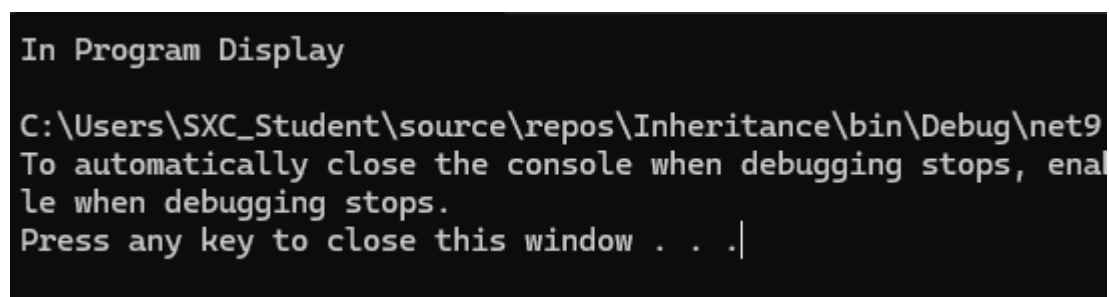
    A a = new C();
    a.Display();

}
}
}

```

Output :

Example 1 :



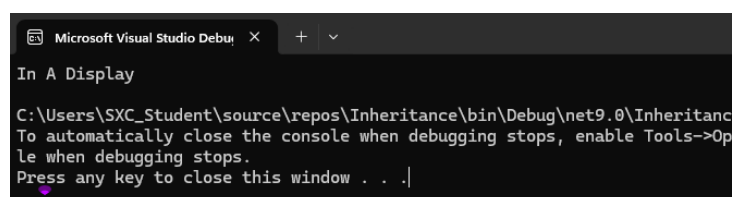
In Program Display

C:\Users\SXC_Student\source\repos\Inheritance\bin\Debug\net9.0\Inheritance.exe

To automatically close the console when debugging stops, enable Tools->Options->Debug->Automatically close the console when debugging stops.

Press any key to close this window . . .|

Example 2 :



Microsoft Visual Studio Debug Console

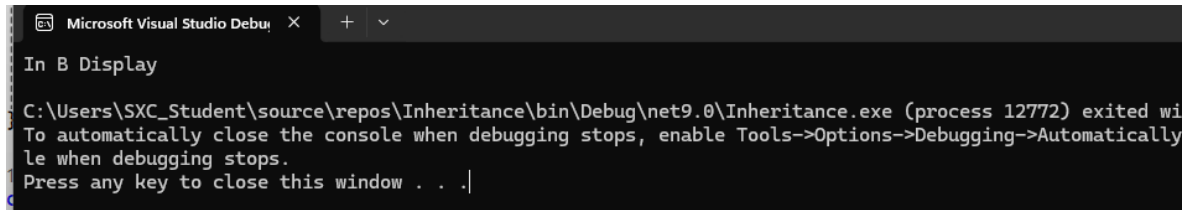
In A Display

C:\Users\SXC_Student\source\repos\Inheritance\bin\Debug\net9.0\Inheritance.exe

To automatically close the console when debugging stops, enable Tools->Options->Debug->Automatically close the console when debugging stops.

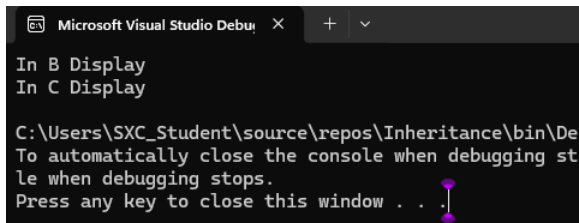
Press any key to close this window . . .|

Example 3 :



```
Microsoft Visual Studio Debug Console
In B Display
C:\Users\SXC_Student\source\repos\Inheritance\bin\Debug\net9.0\Inheritance.exe (process 12772) exited wi
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically
le when debugging stops.
Press any key to close this window . . .|
```

Example 4 :



```
Microsoft Visual Studio Debug Console
In B Display
In C Display
C:\Users\SXC_Student\source\repos\Inheritance\bin\De
To automatically close the console when debugging st
le when debugging stops.
Press any key to close this window . . .|
```

Error and Debug : None

Conclusion : We successfully implemented inheritance and hierarchy using virtual and override.

Practical No : 07

Title : Structures in C#

Date : 09/07/2026

Aim: To demonstrate implementation of Structure in C#

Q1. Write a C# Program to store and retrieve student data. Take the following attributes:

- Roll No
- Name
- Age
- Course
- Marks

Flow Diagram / Logic: None

Program :

```
namespace DataStructure
{
    class Student
    {
        public int rollno;
        public string name;
        public int age;
        public string course;
        public int marks;

        public Student(int roll, string name, int age, string course, int marks)
        {
            this.rollno = roll;
            this.name = name;
            this.age = age;
            this.course = course;
            this.marks = marks;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
```

```

{
    Student[] s = new Student[3];
    int roll;
    string name;
    int age;
    string course;
    int marks;

    for (int i = 0; i < s.Length; i++)
    {

        Console.WriteLine("Enter Record of Student :" + (i + 1));
        Console.WriteLine("Enter Roll No : ");
        roll = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Enter Name : ");
        name = Console.ReadLine();

        Console.WriteLine("Enter Course : ");
        course = Console.ReadLine();

        Console.WriteLine("Enter Age : ");
        age = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Enter Marks : ");
        marks = Convert.ToInt32(Console.ReadLine());

        s[i] = new Student(roll, name, age, course, marks);

    }
}

```

```

        for (int i = 0; i < s.Length; i++)
        {
            Console.WriteLine("-----");

            Console.WriteLine("Student " + (i + 1) + "info :");
            Console.WriteLine(s[i].name);
            Console.WriteLine(s[i].rollno);
            Console.WriteLine(s[i].age);
            Console.WriteLine(s[i].course);
            Console.WriteLine(s[i].marks);
            Console.WriteLine("-----");

        }
    }
}

```

Output :

```

Enter Record of Student :1
Enter Roll No :
1
Enter Name :
Ian Reddy
Enter Course :
Bsc IT
Enter Age :
19
Enter Marks :
99
Enter Record of Student :2
Enter Roll No :
2
Enter Name :
Adrian Dias
Enter Course :
BAMCJ
Enter Age :
19
Enter Marks :
90
Enter Record of Student :3
Enter Roll No :
3
Enter Name :
Shreyas Bhandary
Enter Course :
Bsc IT
Enter Age :

```

```

-----
Student 1info :
Ian Reddy
1
19
Bsc IT
99
-----
Student 2info :
Adrian Dias
2
19
BAMCJ
90
-----
Student 3info :
Shreyas Bhandary
3
19
Bsc IT
78
-----

```

Error and Debug : None

Q2. Write a C# Program to store and retrieve employee data. Take the following attributes:

- Employee Number (eno)
- Name
- Age
- Salary

Retrieve:

- (a) Name and age of employees whose salary is greater than 60000.
- (b) Employees who are above 50 in age.
- (c) Display the age of employee who has maximum salary.

Program :

```
using System.ComponentModel.DataAnnotations;
using System.Globalization;

namespace DataStructure
{
    class emp
    {
        public int eno;
        public string name;
        public int age;
        public int salary;

        public emp(int eno, string name, int age, int salary)
        {
            this.eno = eno;
            this.name = name;
            this.age = age;
            this.salary = salary;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
```

```

emp[] s = new emp[5];
int eno;
string name;
int age;
int salary;

for (int i = 0; i < s.Length; i++)
{

    Console.WriteLine("Enter Record of Employee :" + (i + 1));
    Console.WriteLine("Enter Employee Id : ");
    eno = Convert.ToInt32(Console.ReadLine());

    Console.WriteLine("Enter Name : ");
    name = Console.ReadLine();

    Console.WriteLine("Enter Age : ");
    age = Convert.ToInt32(Console.ReadLine());

    Console.WriteLine("Enter Salary : ");
    salary = Convert.ToInt32(Console.ReadLine());

    s[i] = new emp(enno, name, age, salary);

}

//Display name and age of employee whose salary > 60000
for (int i = 0; i < s.Length; i++)
{
    if (s[i].salary > 60000)

```

```

        {
            Console.WriteLine("Name : " + s[i].name);
            Console.WriteLine("Age : " + s[i].age);
        }

    }

    //Display the age of emp who all are > 50
    for (int i = 0; i < s.Length; i++)
    {

        if (s[i].age > 50)
        {
            Console.WriteLine("Name : " + s[i].name);
            Console.WriteLine("Age : " + s[i].age);
        }
    }

    //Display the age of emp who has max salary
    int max = 0;
    string max_namo = null;
    for (int i = 0; i < s.Length; i++)
    {
        if (s[i].salary > max)
        {
            max = s[i].salary;
            max_namo = s[i].name;
        }
    }
    Console.WriteLine("MAX SALARY BY : " + max_namo + " " + max);

}

}

}

```

Output :

```
Enter Name :  
Joy  
Enter Age :  
88  
Enter Salary :  
67000  
Display name and age of employee whose salary > 60000  
-----  
Name : Saksham Dias  
Age : 19  
Name : Adrian  
Age : 21  
Name : Joy  
Age : 88  
Display the age of emp who all are > 50  
-----  
Name : Joy  
Age : 88  
Display the age of emp who has max salary  
-----  
MAX SALARY BY : Saksham Dias 90000
```

Conclusion: We successfully implemented structure to store and retrieve student and employee details.

Practical No : 08

Title : ASP.NET Razor Pages

Date : 30/07/2026

Aim: To demonstrate implementation of Razor pages.

Q1. Create a website that manages CRUD operation for Employees using Razor Pages MVC.

Flow Diagram / Logic:

Steps :

1. Open Visual Studio and click Create a new project
2. Search for and select ASP.NET Core Web App
3. In the project manager , expand pages folder and new folder name Employees.
4. Inside Employees add a class – “ Employee.cs”
5. Paste the below code into this file.
6. Install the following packages
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.EntityFrameworkCore.Design
Install-Package Microsoft.VisualStudio.Web.CodeGeneration.Design
7. Run the app.

Code :

File : Employee.cs

```
using System.ComponentModel.DataAnnotations;

namespace AshishRazorPage.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        public int ENo { get; set; }

        public int salary { get; set; }

        [EmailAddress]
        public string Email { get; set; }
    }
}
```

Output :

Index

Create New

ENo	salary	Email	
1201	10000	cahayan@cahayan@cahayan.in	Edit Details Delete
1302	45000	tanishka@tanishka@tanishka.in	Edit Details Delete

© 2024 - AshishKamran - [Blog](#)

Create

Employee

ENo

8998

salary

15000

Email

abc@123

Create

[Back to List](#)

Error and Debug : While installing packages , it was throwing error. I later found that it was because the C# used .NET 9 , so I deprecated the packages to .NET.

Conclusion : The above practical clearly demonstrates , basic validation of values in form and CRUD operations.